Computer Architecture
Replacement policies, Types of misses, Multi-level caches, Performance metrics

Ashwath E (AI23BTECH11001)
Suchit Srivastava (AI23BTECH11027)
Nitesh Gokul (AI23BTECH11006)
Donavalli Sai Durga Rishi (AI23BTECH11004)
N. Pawan Surya (AI23BTECH11016)

24 October 2024

1 Cache Replacement Policies

- For Direct mapped caches, replacement policies are redundant as only one specific slot is to be vacated (each index has only one block). So, we must replace that block only.
- For Set Associative (or fully associative) caches, if there exists an invalid block for the set, then simply insert it there. If the set has no invalid entries, the block which gets replaced is called the **victim block**.

The victim block is decided based on different policies, called replacement policies. Some examples are:

FIFO: First in - First Out

- The entries have a *"timestamp"* of when they were put in the cache and the oldest entry gets replaced. In other words, the entries are evicted in order of arrival.
- Hit rate might be low if we need some particular memory location repeatedly which was loaded long ago. So, FIFO does not support temporal locality well.

LRU: Least Recently Used

- The entries have an *"ordering / timestamp"* of when they were last accessed. Unlike FIFO which only considers when the entries were added to the

cache, LRU considers when they were last accessed by the CPU. LRU vacates the the entry which was accessed least recently i.e., in some sense, the entry which has not been accessed in a long while compared to others.

- LRU relies on the fact that an entry which was accessed earliest in the timestamp is most likely to be accessed late in future. Thus, it follows "past is future" rule.
- LRU is simple to implement for 2-way associative caches, Manageable for 4-way associative caches but tremendously difficult to implement higher associativity. LRU can yield better results than FIFO in the sense that it considers that an element loaded long ago might be needed later on if it is repeatedly accessed.

Moreover in LRU linear array traversal, may fill up more entries than necessary. Similarly, it showcases suboptimal performance in:

1. Scanning patterns like grep, search.
2. Streaming Applications.
3. Simplistic sequential array accessing without repeated access for elements.

LFU: Least Frequently Used

The entries have an ordering based on the count of number of access *"frequency count"*. The entry with the least frequency, i.e., with the lowest number of accesses is replaced. So the cache vacates the entry which is least times accessed by the CPU.

This approach can cause problems as the element that is put into the cache just fresh gets instantly replaced if another block tries to use that index/set. Thus, even if one particular entry has not been accessed in a while (but was accessed a lot earlier) will have a higher frequency count, it will stay there if elements repeatedly try to enter that index, as the latest entry will almost always be the next to be replaced.

Thus, LFU is generally not use standalone, but in combination with some other policy like LRU or so.

Random

In the larger order of things, sometimes randomly picking which entry to replace can also yield good results. But since this approach relies on randomness, one can't say with certainty if it will outperform another policy always. Moreover, it does not provide the same performance for repeated execution of the same code.

Randomness works really well in cases of high associativity. Randomness might not be the best policy to apply if the programs have a very particular access pattern where some variable are accessed VERY frequently (vacating these out would lead to high penalties).

Other policies

Some more common policies are combinations of the above like

1. LRU and Random (Pseudo-LRU): Instead of replacing the least recently used block, we choose a block which is NOT-recently used. Among those which are not recently used, we can replace any block randomly. This simplifies the LRU implementation as we need only 1-bit to identify if a block was recently accessed or not, and provides almost similar performance as the full LRU policy.
2. LRU and LFU: We may combine LRU and LFU. We can consider frequency of accesses, but if a block is very recently brought in the cache, we could wait until it becomes not recently accessed.
3. Other combinations of policies are also possible.
4. We can also consider dirty bit when evicting a block. A non-dirty block can be prioritized for eviction compared to a dirty block as dirty blocks take more cycles to replace due to write-back.

2 Types of Misses

The 3C's of cache are

1. Compulsory (miss)
2. Capacity (miss)
3. Conflict (miss)

Compulsory miss/cold start miss

The very first access to any block will be a miss, independent of the cache architecture. This kind of miss is called a compulsory miss. Compulsory misses can be reduced with a larger block size, but it might have other side-effects, as discussed later.

Capacity misses

Due to finite cache size, many entries have to be replaced before their use is over in the program. So the CPU tries to access them after they have been removed from the cache due to some other entry vacating them. These leads to a miss knows as capacity miss.

Conflict (collision) misses

In caches which are not fully associative, there exists competition for residence of entries in a set. This can cause multiple entries trying to access the same index while the rest of the cache is fairly empty. Such types of misses are called conflict misses where the particular index might be full.

In a fully associative cache, we will not have conflict miss. Sometimes it may be difficult to differentiate if a particular miss is capacity miss or conflict miss. In such situation, we see if that miss would have occured in a fully associative cache or not. Those misses which would have happened even in a fully associative cache are capacity misses and remaining are conflict misses (excluding the compulsory misses which are easy to identify).

3 Advantages and disadvantages of various solutions

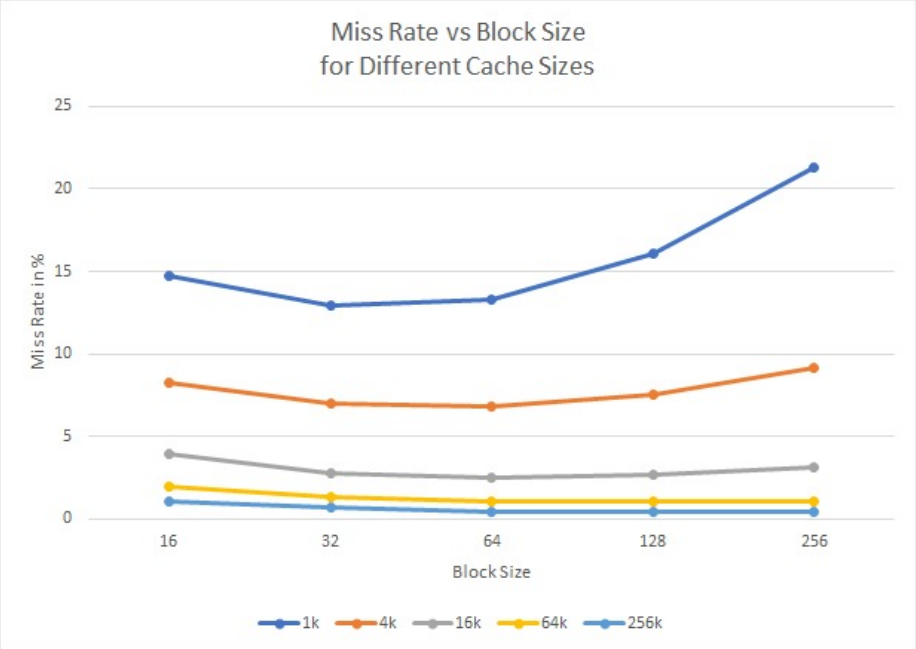
Design Change	Effect on miss Rate	Tradeoff
Increase Cache size	Reduces Capacity misses	May increase access time
Increase Associativity	Reduces Conflict misses	May increase access time
Increase block size	Reduces Compulsory misses	Increased miss penalty, larger sizes increase miss rate due to pollution

Table 1: Various Solutions and their Tradeoffs

Block Size Considerations

- Larger blocks could reduce miss rate due to increased spatial locality.
- But having larger blocks reduces the total number of blocks, which leads to increased competition and a higher miss rate. Cache pollution increases.
- There will be a larger miss penalty as loading a larger block from memory to cache will take more time, which can override the reduced miss rate. Early restart and critical-first-word can help mitigate this problem. Early restart means that we do not wait for the whole block to be loaded from memory, but start execution as the required bytes of the block are loaded in cache. Critical-word-first means we load the required bytes of the block first and then the other portion of the block.

Miss Rate vs Block Size



4 Handling Writes

Writes work somewhat differently. For example on a store instruction, we write the data into the data cache only without changing the main memory. After the write into the cache, memory would have a different value from that in the cache. In such a case, the cache and memory are said to be inconsistent.
The simplest way to keep the main memory and the cache consistent is always to write the data into both the memory and the cache. This scheme is called write-through.

Write-Through

A scheme in which writes always update both the cache and the next lower level of the memory hierarchy, ensuring that data are always consistent between the cache and memory.
With a write-through scheme, every write causes the data to be written to main memory. These writes will take a long time. The solution to use this problem is to use a write buffer.

Write Buffer

A write buffer stores the data while they are waiting to be written to memory. After writing the data into the cache and into the write buffer, the processor can continue execution. If write to main memory completes, the entry in the write buffer is freed.

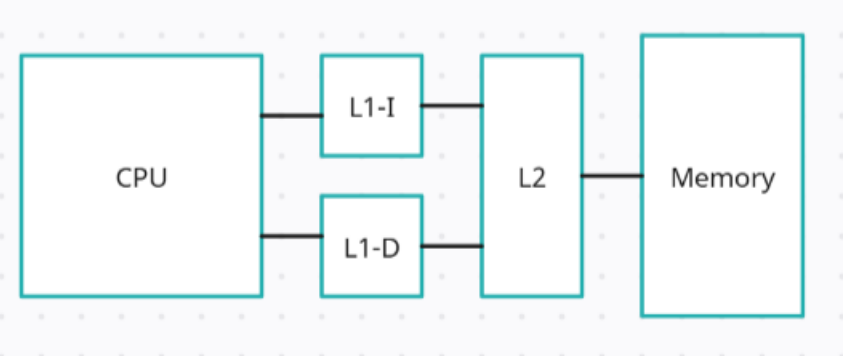
- To reduce the occurrence of stalls which happens when the writes occur in bursts, processors usually increase the depth of the write buffer beyond a single entry.

The alternative to a write-through scheme is a scheme called write-back.

Write-Back

In a write-back scheme, when a write occurs, the new value is written only to the block in the cache. The modified block is written to the lower level of the hierarchy when it is replaced. The write-back scheme is however, more complex to implement than write-through.

5 Multi-level Cache



- Many modern CPUs have different levels of cache to speed up their performance.
- Commonly, there are two caches, the L1-cache and the L2-cache. The L1-cache is adjacent to the CPU and extremely fast but small in size while the L2-cache acts as a mediator between the L1-cache and the Memory.
- The L2-cache is bigger in size but in turn a little slower than L1-cache yet still remarkably faster than the memory.

Often the L1-caches are separated into L1-I cache and L1-D cache which is elaborated upon in the next section. Some of the recent CPUs come with 3 or 4 levels of cache.

Multilevel Cache Considerations

- The L-1 cache focuses on minimal hit time, and is typically smaller than a single cache.
- The L-2 cache focuses on low miss rate to reduce main memory access. Hit time has less impact.
- The block size of L-1 should be same or smaller than that of L-2, for practical purposes.

6 Unified vs Split Caches

Unified Cache

In a unified cache, the instructions and data are both stored in the same cache. One can evict the other if needed.

Split Cache

- There are two types of caches termed I-cache and D-cache respectively, with I-cache storing instructions and D-cache storing data.
- I-cache and D-cache can have different configurations.
- This is useful as Instructions tend have remarkably high spatial locality when compared to typical data accessed by the program.
- Individually, the 2 caches are smaller than the unified cache (typically).

Most processors use separate caches for instructions and data at the L1 level, but unified cache at the L-2 or other levels.

7 Measuring Cache Performance

CPU time can be divided into the clock cycles that the CPU spends executing the program and the clock cycles that the CPU spends waiting for the memory system. Thus,
CPU Time = (CPU execution clock cycles + Memory-stall clock cycles) x Clock cycle time

The memory-stall clock cycles come primarily from cache misses. So, The Memory-stall clock cycles can be defined as the sum of the stall cycles coming from reads plus those coming from writes:

- Memory-stall clock cycles = (Read-stall cycles + Write-stall cycles)

Read-stall cycles are defined as:

- Read-stall cycles = (Reads / Program) x Read miss rate x Read miss penalty

Write-stall cycles are defined as:

- ((Writes/Program) * Write miss rate x Write miss penalty) + Write buffer stalls

With all the above results and also some assumptions like neglecting the write buffer etc.. we get the below results.

Memory-stall clock cycles = (Memory accesses / Program) x Miss rate x Miss penalty

Memory-stall clock cycles = (Instructions / Program) x (Misses / Instruction) x Miss penalty

8 Average Access Time

Hit time is also important for performance.
Average Memory Access Time(AMAT) is calculated as:

- AMAT = Hit time + (Miss rate * Miss penalty)

9 Performance Summary

- When CPU performance is increased, the miss penalty becomes more significant.
- If the base CPI is reduced, a greater proportion of time is spent on memory stall cycles.
- If the clock rate is increased, the memory stalls account for more CPU cycles.

10 Cache Overheads

Cache overheads are the extra costs associated with managing a cache memory system in a computer. Few types of cache overheads are:

Area Overheads

These overheads are caused by tag bits, valid/dirty/LRU bits, comparators and multiplexers.

Delay Overheads

These overheads are caused mainly because of delays caused by matching tags and then accessing data. Write-back also causes an additional time delay.

Power/Energy Overheads

These overheads are caused by the power required to maintain tags, match tags, etc. There is also a power wastage if blocks are not used.